UNDEF

FADENA
Facultad de Defensa Nacional

Facultad de Defensa Nacional

Carrera: Lic. en Ciberdefensa

Materia

Sistemas Operativos I

Unidad 3 / Semana 9

Año: 2025

Contenidos

Herramientas de virtualización. Computación en la nube.

Índice

1. Herramientas de virtualización 3
1.1 IaC - Infraestructura como código 3
Comparación entre programación declarativa e imperativa 4
Ventajas de la programación declarativa 4
Ventajas de la programación imperativa 4
Desventajas de la programación declarativa 4
Desventajas de la programación imperativa 4
1.2 Vagrant 4
2. Computación en la nube (cloud computing) 5
2.1 ¿Qué es la computación en la nube? 5
2.2 Ventajas de usar computación en la nube 6
2.3 Tipos de nube 7
2.3.1 Nube Pública 7
2.3.2 Nube Privada 7
2.3.3 Nube Híbrida 7
2.3.4 Multinube 7
2.4 Servicios de Computación en la nube 7
2.4.1 IaaS 7
2.4.2 PaaS 7
2.4.3 SaaS 8
2.5 Virtualización usando contenedores 8
2.5.1 Docker 10
3. Sistemas operativos en red **11**
a. Servidor como programa o aplicación 11
b. Servidor como hardware (computadora) 12
c. Sistema Operativo de Red (NOS - Network Operating System) o de servidor 12
d. Sistema Operativo de Servidor 13
3.1 Servidor de red 14
3.1.1 ¿Qué diferencia a un servidor de una computadora cliente? 14
3.1.2 Funciones y usos de los servidores de red 15
4. Sistema operativo Ubuntu Server 15
3.1 Actividad S9-A1 16

1. Herramientas de virtualización

Hemos visto el rol fundamental que tienen los hipervisores en la virtualización de computadoras de todo tipo. Con la evolución de la tecnología de virtualización, se han creado herramientas que aumentan el control y la productividad de ambientes virtualizados, alcanzando la automatización de muchas de las tareas que se hacían en forma manual.

Con el auge de la virtualización y luego la computación en la nube, los administradores de sistemas necesitaban configurar servidores manualmente en forma masiva y con el riesgo de producir errores. Esto impulsó lo que se conoce como *Infraestructura como Código*.

1.1 IaC - Infraestructura como código

La **infraestructura como código** (***IaC*** del inglés ***Infrastructure as Code***) permite gestionar y preparar la infraestructura a través del código, en lugar de hacerlo mediante procesos manuales.

Con este tipo de infraestructura, se crean archivos de configuración que contienen las especificaciones que ésta necesita, lo cual facilita la edición y la distribución de las configuraciones. Asimismo, garantiza que siempre se prepare el mismo entorno. La infraestructura como código codifica y documenta sus especificaciones para facilitar la gestión de la configuración, y le ayuda a evitar los cambios ad hoc y no documentados.

Como la configuración se basa en archivos, el control de versiones es un aspecto importante de la *infraestructura como código* que se debería aplicar a los archivos de configuración, al igual que a cualquier otro archivo de código fuente del software. La implementación de la infraestructura como código también permite dividirla en elementos modulares que se combinarán de distintas maneras mediante la automatización.

Si se automatiza la preparación o aprovisionamiento de la infraestructura con la *infraestructura como código*, los desarrolladores no tendrán que preparar ni gestionar manualmente los servidores, los sistemas operativos, el almacenamiento ni ningún otro elemento cada vez que desarrollen o implementen una aplicación.

Hay dos maneras de abordar la *IaC:* mediante un **enfoque declarativo** o un **enfoque imperativo**.

El ***enfoque declarativo*** define el estado final deseado de los sistemas, lo cual incluye los recursos que se necesitan y las propiedades que deben tener dichos sistemas, y la herramienta de infraestructura como código se encargará de crearlo y configurarlo.

En cambio, el **enfoque imperativo** define los comandos específicos para lograr la configuración deseada, es decir “el cómo” se deben crear, los cuales se deben ejecutar en el orden correcto.

Muchas herramientas de la *infraestructura como código* utilizan un enfoque declarativo y prepararán la infraestructura deseada de manera automática. Por lo tanto, si realiza modificaciones en el estado deseado, una herramienta de IaC declarativa las implementará por usted; pero si utiliza una herramienta imperativa, deberá resolver la manera de aplicar esos cambios.

La mayoría de las herramientas de la infraestructura como código pueden operar con ambos enfoques, pero tienden a dar prioridad a alguno de ellos.

Comparación entre programación declarativa e imperativa

Ventajas de la programación declarativa

- Requiere menos habilidades de codificación
- Control de cada paso del proceso
- Altamente repetible

Ventajas de la programación imperativa

- Ideal para tareas sencillas o puntuales
- Se adapta bien a los cambios de configuración
- Sigue un enfoque intuitivo, paso a paso

Desventajas de la programación declarativa

- Menos control sobre el proceso
- Requiere muchos conocimientos de programación
- Puede complicar en exceso tareas sencillas

Desventajas de la programación imperativa

- Menos idempotente (si se repite no garantiza el mismo resultado)
- Puede ser más difícil de conceptualizar
- El nivel de detalle lo hace más propenso a errores

Algunos ejemplos de herramientas de IaC populares que utilizan el paradigma de programación declarativa son *Terraform, Puppet, Ansible, Salt y CloudFormation*.

Chef es una herramienta popular de la IaC imperativa, y tanto *Ansible* como *Salt* también tienen soporte para programación imperativa.

1.2 Vagrant

Vagrant es un software de código abierto que nos permite crear y mantener entornos de desarrollo portables, puede trabajar con VMware, VirtualBox, Hyper-V, KVM, AWS e incluso también con contenedores de Docker.

Por lo tanto, es una opción ideal para simplificar y automatizar la configuración de estos software de virtualización.

Vagrant está escrito en lenguaje Ruby, pero se permite utilizar otros lenguajes de programación sin ningún problema como por ejemplo python, o bash.

Se basa en unos archivos de configuración denominados *Vagrantfiles*, estos archivos pueden ser compartidos entre desarrolladores para desplegar en sus equipos máquinas virtuales ya creadas.

La utilización de Vagrant es útil en equipos de desarrollo compuestos por varias personas y basadas en diferentes locaciones puesto que nos asegura que todos y cada uno de los integrantes trabajen con el mismo entorno de desarrollo.

Este tema se profundiza en el libro ***VirtualBox para dummies***, especialmente en el *capítulo 2*, que está publicado en el campus, en donde podrá ver ejemplos de uso de esta herramienta.

2. Computación en la nube (cloud computing)[1]

La computación en la nube permite acceder a recursos y productos informáticos a través de la Web. Estos recursos incluyen herramientas para desarrolladores, aplicaciones comerciales, servicios de computación, almacenamiento de datos y soluciones de redes.

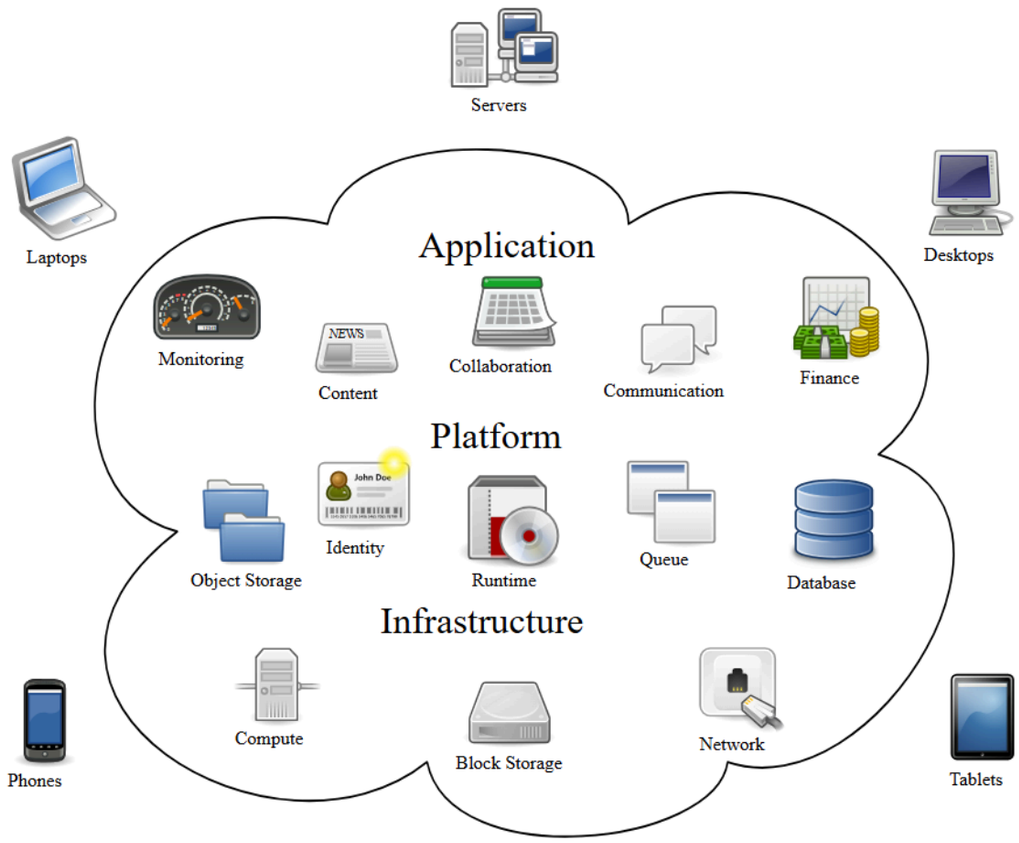
Estos servicios en la nube pueden alojarse en un centro de datos del proveedor o gestionarse en entornos locales en el centro de datos propios.

2.1 ¿Qué es la computación en la nube?

En términos sencillos, Cloud Computing permite *alquilar* tecnologías de la información en lugar de *comprarlas*. Las empresas pueden acceder a su capacidad de computación a través de Internet, o la nube, y pagar según el uso. Los servicios en la nube incluyen servidores, almacenamiento, bases de datos, redes, software, análisis e inteligencia empresarial.

La computación en la nube ofrece velocidad, escalabilidad y flexibilidad, permitiendo que los negocios se desarrollen e innoven.

[1] Fuente: https://www.oracle.com/ar/cloud/what-is-cloud-computing/



https://en.wikipedia.org/wiki/Cloud_computing

2.2 Ventajas de usar computación en la nube

La computación en la nube impulsa a las empresas a trasladarse a la nube por la agilidad, escalabilidad y flexibilidad necesarias para competir. La explosión de datos y el aumento de costos de almacenamiento requieren nuevas herramientas de análisis. Los servicios de almacenamiento en la nube ayudan a las empresas a afrontar estos desafíos, ofreciendo valor comercial y reduciendo costos. La nube pública y la nube privada proporcionan alternativas superiores a la tecnología tradicional.

Beneficios específicos de la computación en la nube:

- Costo: Reducción de gastos de capital
- Velocidad: Aprovisionamiento instantáneo para desarrollo y pruebas
- Escala global: escala flexible
- Productividad: Mayor colaboración y rendimiento predecible
- Rendimiento: Mejor relación precio-rendimiento para cargas de trabajo nativas de la nube
- Confiabilidad: Sistemas distribuidos y tolerantes a fallos

2.3 Tipos de nube

Existen tres tipos principales de nube: nube pública, nube privada y nube híbrida. Cada tipo ofrece distintos niveles de gestión y seguridad.

2.3.1 Nube Pública

En una nube pública, toda la infraestructura se ubica en las instalaciones del proveedor, prestando servicios y espacio en la nube a través de Internet. Los clientes pueden agregar usuarios y potencia según necesidad, compartiendo la infraestructura con otros inquilinos.

2.3.2 Nube Privada

La nube privada es de uso exclusivo para una organización y puede alojarse en su ubicación o en el centro de datos del proveedor. Ofrece un nivel más alto de seguridad y control.

2.3.3 Nube Híbrida

Una nube híbrida combina nubes públicas y privadas, alojando aplicaciones críticas en servidores propios y aplicaciones secundarias en el entorno del proveedor.

2.3.4 Multinube

La principal diferencia entre nube híbrida y multinube es el uso de varios dispositivos de computación en la nube y el almacenamiento en una sola arquitectura.

2.4 Servicios de Computación en la nube

Existen tres tipos principales de servicios en la nube, son:

IaaS (Infraestructura como servicio), PaaS (Plataforma como servicio) y SaaS (Software como servicio).

2.4.1 IaaS

La infraestructura como servicio (IaaS) proporciona servicios básicos de infraestructura según demanda, con el proveedor gestionando la capacidad de red, almacenamiento y computación y el cliente gestionando desde el SO.
Ejemplos: AWS EC2, Google Compute Engine, Microsoft Azure Virtual Machines.

2.4.2 PaaS

La plataforma como servicio (PaaS) ofrece, además de la IaaS, herramientas para desarrolladores sin necesidad de invertir en infraestructura subyacente. Permite integrar características innovadoras en aplicaciones, como inteligencia artificial y blockchain.

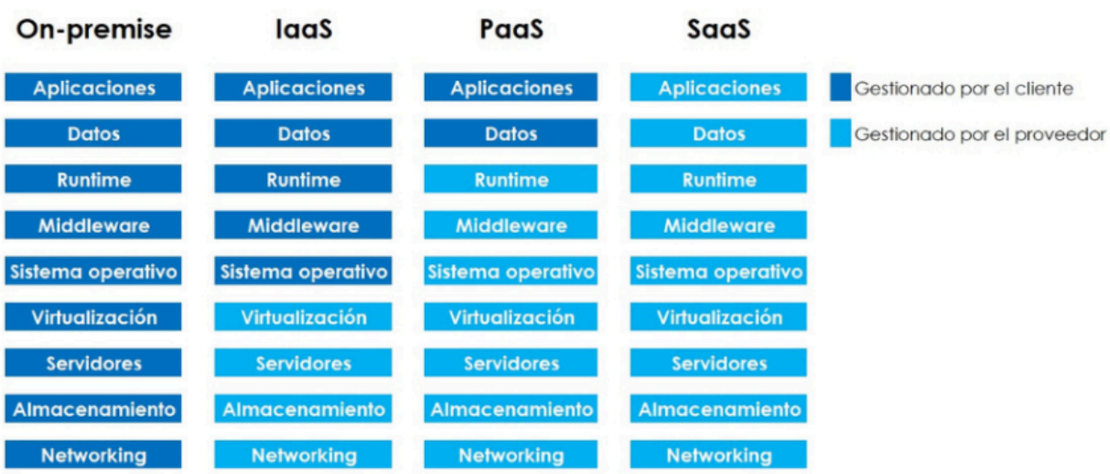
Para mejorar la productividad, las soluciones PaaS deben incluir componentes de programación listos para usar que faciliten la integración de nuevas características como IA, chatbots, blockchain e IoT. Además, deben ofrecer herramientas para analistas, usuarios finales y administradores de TI, como analítica de big data, gestión de contenido, bases de datos, sistemas y seguridad.

2.4.3 SaaS

El software como servicio (SaaS) es un modelo en el que el proveedor aloja las aplicaciones del cliente, accesibles a través de Internet. El SaaS permite a las empresas ponerse en marcha rápidamente con tecnología innovadora, ajustando servicios según necesidades.

Para muchas empresas, SaaS es la solución ideal porque les permite ponerse en marcha rápidamente con la tecnología más innovadora disponible. Los clientes pueden ampliar o reducir los servicios en función de las fluctuaciones de las cargas de trabajo, agregando más servicios o funciones a medida que crezcan.

Ejemplos : ERPs, CRMs, etc.



2.5 Virtualización usando contenedores[2]

Cuando la virtualización se realiza a nivel del *sistema operativo,* se obtienen los contenedores (o virtualización basada en contenedores) que son el resultado de aplicar una capa de virtualización sobre el núcleo del sistema operativo que permite que existan múltiples instancias aisladas de espacios de usuario, en lugar de solo uno.

Estas instancias se denominan contenedores (utilizado LXC, Docker), pero también reciben otros nombres, como zonas (Solaris), servidores privados virtuales (OpenVZ), particiones/entornos virtuales (VEs), núcleo virtual (DragonFly BSD) o jaulas (jail FreeBSD).

En 2008 se ofrece para sistemas Linux, LinuX Containers (LXC), que utilizando namespaces y cgroups del kernel, permite crear entornos de gestión de

[2] Nickoloff, J.; Kuenzli, S. (2019). Docker in Action. Manning.

contenedores y, en 2013 y después de algunos años de desarrollo, se publica Docker, que permite gestionar el ciclo de vida de los contenedores de forma sencilla en comparación con las herramientas anteriores teniendo gran aceptación por los desarrolladores y un crecimiento exponencial en su uso, ya que permite dentro de los contenedores, empaquetar aplicaciones aisladas entre sí con capacidad total de ejecución y, además, permite que puedan ser integradas en los flujos de integración/distribución continua generando un único flujo desde el desarrollo de aplicaciones hasta su puesta en producción.

Docker usó como entorno de ejecución por defecto *LXC*. Sin embargo, más tarde fue reemplazado por *libcontainer* permitiendo utilizar otras tecnologías de aislamiento (distintas a las de LXC) y poder acceder directamente a las API del kernel del sistema operativo reduciendo sus dependencias de librerías y aumentando la eficiencia del conjunto.

No son los únicos sistemas basados en la virtualización del SO. Apache Mesos abstrae la CPU, la memoria, el almacenamiento y otros recursos informáticos de las máquinas (físicas o virtuales), lo que permite que se pueda construir un sistema distribuido elástico y tolerante a fallas y que se ejecuten fácilmente de manera efectiva. Este sistema está especialmente orientado a entornos de Big Data.

Windows también ha desarrollado su propio ecosistema (Windows Server Container / Hyper V Container). No obstante, la versión de Docker para Windows que utiliza la propia infraestructura de Windows es la referencia actual en el SO Windows.

Probablemente, en el mundo de los contenedores, el hecho más significativo después de la creación de estos fue (en 2014) la apertura por parte de *Google* del proyecto ***Kubernetes*** (también conocida como K8s) como una plataforma portátil, extensible y de código abierto que permite administrar cargas de trabajo y servicios en contenedores y facilita una configuración declarativa como la automatización. Esta plataforma se ha convertido en el referente en la industria para el despliegue basado en contenedores, la gestión de cargas y la escalabilidad horizontal y es uno de los entornos más utilizados en los procesos de integración contínua del desarrollo de software.

Los expertos clasifican los sistemas de contenedores como:

- **Contenedores de infraestructura**, como LXC/LXD, donde se permite ejecutar múltiples instancias de sistemas operativos de manera aislada permitiéndole acceder a los recursos (CPU, network, I/O) en forma similar a una máquina virtual pero más rápido y ligero. Ejemplos de estos sistemas son LXC y LXD.

- **Contenedores de procesos o aplicaciones**, como systemd-nspawn o *Docker* (por defecto en todo Linux), que ofrecen un sistema de encapsulado de aplicaciones, permiten acelerar su desarrollo y distribución eliminando las diferencias de entorno entre producción y desarrollo y facilitan la migración del entorno.

- **Contenedores sandbox**, como FireJail, Bubblewrap, nsroot, nsjail, que permiten el aislamiento mediante un entorno donde se tiene acceso restringido a recursos del sistema operativo o datos del usuario (sandbox).

2.5.1 Docker

Vamos a enfocarnos en *Docker* en este caso.



Docker es una plataforma abierta para el desarrollo, empaquetado y ejecución de aplicaciones de forma que se puedan poner en producción o compartir más rápidamente y que separa estas de la infraestructura. Es por ello que el costo es mucho menor en recursos (básicamente espacio de disco, CPU y arranque) y permite que los desarrolladores puedan disponer de todo tal y como lo ha dejado el anterior desarrollador sobre su infraestructura. Esto significará menos tiempo para probar y acelerará el despliegue acortando de forma significativa el ciclo entre que se escribe el código y pasa a producción. Docker emplea una plataforma (contenedor) de virtualización ligera con flujos de trabajo y herramientas que la ayudan a administrar e implementar las aplicaciones y proporcionar una forma que casi cualquier aplicación se podrá ejecutar de forma segura en un contenedor aislado.

Este aislamiento y seguridad permiten ejecutar muchos contenedores de forma simultánea en el host y, dada la naturaleza (ligera) de los contenedores, todo ello se ejecuta sin la carga adicional de un hipervisor (que sería la otra manera de gestionar estas necesidades compartiendo la VM), lo que significa se pueden obtener mejores prestaciones y utilización de los recursos. Es decir, con una VM, cada aplicación virtualizada incluye no solo la aplicación, que puede ser decenas de Mbytes –binarios + bibliotecas–, sino también el sistema operativo guest, que pueden ser varios Gbytes; en cambio, en Docker solo será la aplicación y sus dependencias, que se ejecutan como un proceso aislado en el espacio de usuario del SO host, compartiendo el kernel con otros contenedores.

Por tanto, tiene el beneficio del aislamiento y la asignación de recursos de las máquinas virtuales, pero es mucho más portátil y eficiente transformándose en el entorno perfecto para apoyar el ciclo de vida del desarrollo de software, test de plataformas, entornos, etc.

El entorno está formado por dos grandes componentes: **Docker** (es la plataforma de virtualización –contenedor–) y **Docker Hub** (una plataforma SasS que permite obtener y publicar/gestionar contenedores ya configurados). En su arquitectura, Docker utiliza una estructura cliente-servidor en la que el cliente (CLI Docker) interactúa con el daemon Docker, que hace el trabajo de la construcción, ejecución y distribución de los contenedores de Docker. Tanto el cliente como el

daemon se pueden ejecutar en el mismo host, o se puede conectar un cliente a un daemon de Docker remoto. El cliente Docker y el servicio se comunican mediante sockets o una API RESTful.

Para tener más detalles sobre la arquitectura se puede consultar https://docs.docker.com/get- started/overview/.

Su instalación es sumamente fácil, ya que Docker mantiene los binarios para cada distribución y el proceso de instalación se puede consultar de la página web del desarrollador ya sea para Linux, MacOS o Windows.

La forma más simple de verificar su funcionamiento es ejecutar un contenedor llamado *hello-world* simplemente poniendo *docker run hello-world* que se descargará una **imagen del contenedor del Docker-Hub,** lo pondrá en marcha y mostrará una pantalla de bienvenida.

Es importante el concepto de **imagen de un contenedor** y el **contenedor** propiamente dicho: la imagen contiene todos los elementos y configuraciones para crear el contenedor pero *no es el contenedor*. El contenedor se crea a partir de la imagen y puede ser modificado, reiniciado o parado, pero los cambios que se realicen en él no se reflejarán en la imagen que *solo es de lectura*. Si se quieren guardar los cambios en un contenedor se deberá hacer una nueva imagen con el nuevo contenedor (commit).

3. Sistemas operativos en red

Primero definiremos el término servidor, el cual tiene dos acepciones principales, aunque relacionadas:

a. Servidor como programa o aplicación

Originalmente, un **servidor** se refería a un **programa o aplicación informática** que ofrece servicios a otras aplicaciones, conocidas como **clientes**.

Por ejemplo:

- **Servicios de archivos:** Permiten a los usuarios almacenar y acceder a archivos de forma centralizada.
- **Servicios de aplicaciones:** Ejecutan tareas específicas en beneficio directo del usuario final.

Es importante destacar que una misma computadora puede funcionar simultáneamente como cliente y como servidor.

b. Servidor como hardware (computadora)

Por extensión, también se denomina **servidor** a la **computadora donde se ejecuta uno o varios de estos programas de servicio**. Estas máquinas están diseñadas para proveer recursos a los clientes en una red.

Ahora aclararemos el significado de sistema operativo de red (NOS, por sus siglas en inglés) y sistema operativo de servidor, los cuales a menudo se usan indistintamente, especialmente en la terminología moderna, originalmente tenían matices distintos que aún son relevantes para comprender su evolución y función.

c. Sistema Operativo de Red (NOS - Network Operating System) o de servidor

Históricamente, el término **Sistema Operativo de Red** (NOS) se refería más ampliamente a un **software diseñado para permitir que múltiples computadoras compartan recursos y se comuniquen dentro de una red**. En sus inicios, esto podía aplicarse a diferentes arquitecturas de red, no necesariamente solo a servidores dedicados.

Características clave del NOS en su concepción original:

- **Enfoque en la conectividad:** Su función primordial era facilitar la comunicación y el intercambio de datos entre computadoras.
- **Gestión de recursos distribuidos:** Permitía a los usuarios acceder a recursos como impresoras o archivos que estaban en diferentes máquinas de la red.
- **Arquitecturas variadas:** Podía soportar modelos de red de igual a igual (peer-to-peer) donde cada estación de trabajo podía compartir recursos, o arquitecturas cliente-servidor con un servidor dedicado.
- **Ejemplos históricos:** Windows Server, Linux, Unix, Novell netware (fue uno de los primeros sistemas operativos de red, muy popular en los años 90, ahora en desuso) como se mencionó anteriormente.

  En la actualidad, todos los sistemas operativos ya tienen capacidad de conexión de redes y compartir recursos. No obstante, los sistemas operativos de servidor permiten compartir los recursos en forma más eficiente y segura.

d. Sistema Operativo de Servidor

Un **Sistema Operativo de Servidor** es una **versión especializada de un sistema operativo diseñada específicamente para funcionar en un servidor de red**. Su propósito es gestionar los recursos del servidor y proporcionar servicios de manera eficiente, segura y escalable a múltiples clientes conectados a la red.

Características clave de un Sistema Operativo de Servidor:

- **Optimizado para el rendimiento y la estabilidad:** Diseñado para funcionar 24/7 con alta disponibilidad, minimizando caídas y maximizando el rendimiento bajo cargas pesadas.
- **Funcionalidades de servidor integradas:** Incluye herramientas y servicios específicos para funciones de servidor, como:
    - **Gestión de usuarios y grupos:** Directorios como Active Directory (Windows Server) o LDAP (Linux).
    - **Servicios de archivos:** Compartir y gestionar archivos de forma centralizada.
    - **Servicios de impresión:** Controlar y compartir impresoras en la red.
    - **Servicios web (HTTP/HTTPS):** Alojar sitios web y aplicaciones web.
    - **Servicios de DNS**, resolución de nombres
    - **Servicio de DHCP**, de asignación de ip dinámicas
    - **Servicios de base de datos:** Soportar sistemas de gestión de bases de datos.
    - **Servicios de correo electrónico:** Alojar servidores de correo.
    - **Herramientas de seguridad:** Firewalls, auditoría de seguridad, políticas de acceso.
    - **Virtualización:** Capacidades para crear y gestionar máquinas virtuales.
- **Mayor seguridad:** Ofrece características de seguridad robustas para proteger los datos y los servicios.
- **Escalabilidad:** Capacidad para manejar un gran número de usuarios y solicitudes simultáneas, y para crecer según las necesidades de la red.
- **Ejemplos actuales:** Windows Server (todas sus versiones), varias distribuciones de Linux optimizadas para servidor (Ubuntu Server, Red Hat Enterprise Linux, CentOS Stream), y versiones de UNIX.

3.1 Servidor de red

Los **servidores de red** son computadoras o dispositivos dedicados que actúan como un **centro neurálgico** en una red. Su función principal es **administrar y ofrecer recursos compartidos** a otras computadoras (clientes) conectadas a la red. Estos recursos pueden incluir:

- **Espacio de almacenamiento:** Para guardar archivos y documentos.
- **Acceso a hardware:** Como impresoras o dispositivos de almacenamiento externos.
- **Servicios de software:** Tales como correo electrónico, bases de datos o aplicaciones compartidas.

3.1.1 ¿Qué diferencia a un servidor de una computadora cliente?

A diferencia de las computadoras cliente, los servidores están diseñados para operar de forma **continua y fiable**, por lo que se enfocan en la **estabilidad, seguridad y alto rendimiento**. Para lograr esto, suelen integrar **componentes de hardware más potentes y especializados**, como:

- **Unidades Centrales de Procesamiento (CPU):** Con mayor capacidad para ejecutar múltiples tareas simultáneamente.
- **Chipsets:** Que optimizan la comunicación entre los componentes del servidor.
- **Memoria RAM:** En grandes cantidades para manejar eficientemente las solicitudes de los clientes.
- **Sistemas de disco:** Con configuraciones que priorizan la velocidad y la redundancia de datos (por ejemplo, RAID).

Estos componentes están pensados para soportar operaciones más complejas y exigentes. Además, los servidores suelen ejecutar **sistemas operativos diseñados específicamente para entornos de red**, que ofrecen mayor robustez y seguridad, y están configurados para funcionar las 24 horas del día sin interrupción.

3.1.2 Funciones y usos de los servidores de red

Servir solicitudes de red

La principal función de un servidor de red es recibir y responder a las solicitudes de los ordenadores cliente a través de la red. Un ejemplo de ello es cuando un usuario hace clic en un enlace y el servidor de red responde con la página solicitada.

Almacenamiento y procesamiento de datos

Los servidores permiten almacenar, acceder y transferir grandes cantidades de datos. Además, pueden ofrecer servicios como bases de datos para el procesamiento y almacenamiento de la información, facilitando el crecimiento empresarial.

Protección contra ataques malignos

Los servidores de red también protegen los sistemas contra posibles ataques de hackers. Monitorean las solicitudes de datos y bloquean accesos sospechosos para evitar la transmisión de malware o accesos no autorizados.

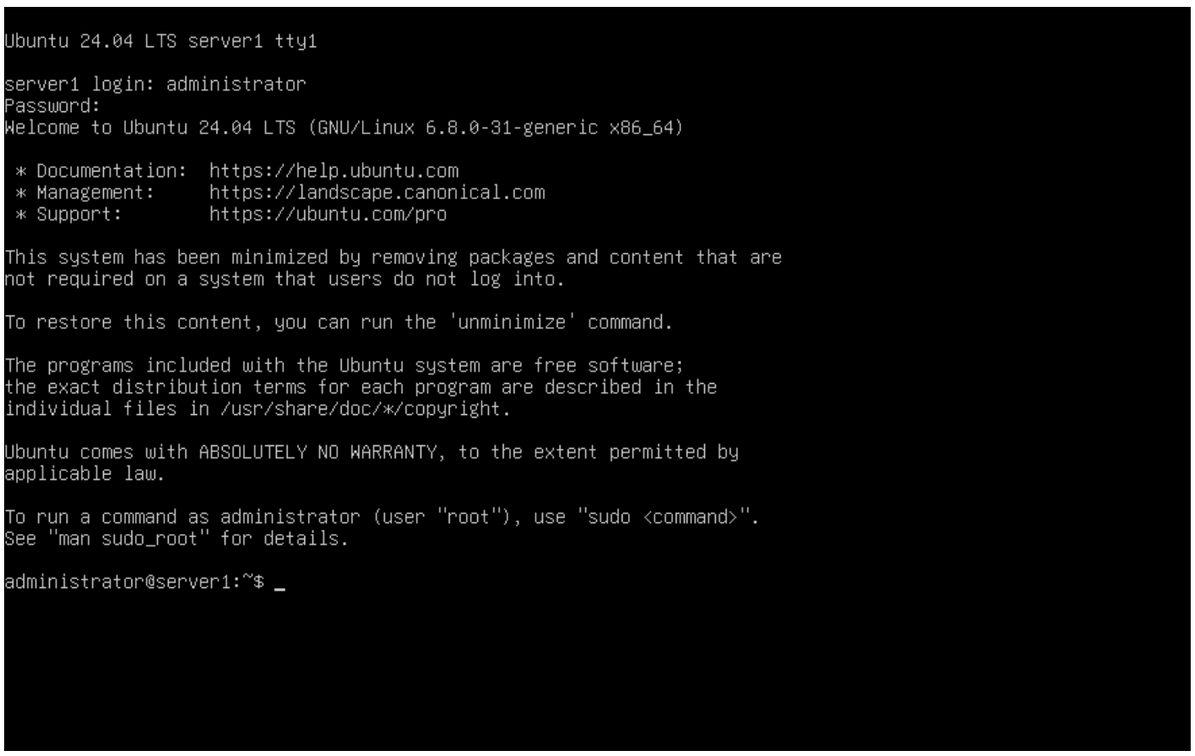
Copias de seguridad de datos

Las aplicaciones sensibles requieren servidores que ofrezcan copias de seguridad automáticas. Esto asegura que los datos se encuentren protegidos, incluso en caso de fallos del servidor principal.

4. Sistema operativo Ubuntu Server

Ubuntu es una de las distribuciones de Linux más extendidas. Brinda **facilidad de uso**, su probada **estabilidad** y el **sólido soporte de una comunidad activa**. Existen dos variantes principales para esta distribución: **Ubuntu Desktop**, diseñada para usuarios finales, y **Ubuntu Server**, optimizada para entornos de servidor. Ambas versiones, si bien comparten un núcleo común, están ajustadas para satisfacer necesidades y casos de uso específicos.

Ubuntu Server no incluye una interfaz gráfica de usuario (GUI), lo que reduce la sobrecarga del sistema y mejora el rendimiento. Se puede instalar una GUI pero a costa de reducir el rendimiento del sistema.



Pantalla de login de Ubuntu Server

3.1 Actividad S9-A1



Actividad S9-A1

Realice la actividad identificada como **S9-A1** en la sección de actividades del campus.